YACHAY TECH

ESCUELA DE CIENCIAS MATEMÁTICAS Y COMPUTACIONALES

UNIVERSIDAD DE INVESTIGACIÓN DE TECNOLOGÍA EXPERIMENTAL YACHAY TECH

School of Mathematical and Computational Sciences

Course: Analysis of Algorithms (AoA)

Laboratory Report 09: Greedy Technique

Prim's Algorithm, Dijkstra's Algorithm, and Huffman Coding

Author: Student Laboratory Report
Course: Analysis of Algorithms
Institution: Yachay Tech University

October 26, 2026

Abstract

This report presents an exhaustive theoretical and practical analysis of **Laboratory 09: Greedy Technique** for the Analysis of Algorithms course at Yachay Tech University. The practice investigates three foundational problems in combinatorial optimization: (1) Minimum Spanning Tree (MST) construction using Prim's algorithm on a weighted undirected graph with full priority queue tracing; (2) design and implementation of Prim's algorithm in standard C and modern C++; (3) analytical step-by-step resolution of the Single-Source Shortest-Paths (SSSP) problem via Dijkstra's algorithm across two distinct instances (a 5-vertex directed graph and a 12-vertex interconnected network); (4) computational implementation of Dijkstra's algorithm in C and C++; and (5) analysis, binary tree synthesis, string encoding, and sequence decoding using optimal Huffman Coding. All software implementations were compiled under the C11 and C++17 standards with strict optimization and warning flags, confirming exact agreement with analytical derivations.

---

1 Introduction and Theoretical Framework

The **Greedy Technique** is a foundational algorithmic design paradigm applied to solve optimization problems. Unlike global strategies such as dynamic programming or brute-force search, a greedy algorithm makes the locally optimal choice at each sequential step with the expectation that these immediate decisions will lead to a globally optimal solution.

1.1 Conditions for Greedy Optimality

A greedy strategy is guaranteed to find a global optimum if and only if the underlying problem exhibits two primary characteristics:

1. **Greedy Choice Property:** A globally optimal solution can be assembled incrementally by making locally optimal (greedy) choices without reconsidering earlier choices (no backtracking).
2. **Optimal Substructure:** An optimal solution to the global problem incorporates optimal solutions to its constituent subproblems.

1.2 Prim's Algorithm for Minimum Spanning Trees (MST)

Given a connected, weighted, undirected graph $G = (V, E, w)$, a Minimum Spanning Tree is an acyclic subgraph $T = (V, E_T)$ spanning all vertices V while minimizing the aggregate edge weight:

$$W(T) = \sum_{e \in E_T} w(e) \tag{1}$$

Prim's algorithm initializes the tree with an arbitrary root vertex v_0 ($V_T = \{v_0\}$) and iteratively extends V_T by selecting the vertex $u \in V \setminus V_T$ that minimizes the connection cost to any vertex currently in V_T. The remaining vertices $V \setminus V_T$ are organized in a priority queue where each vertex is tracked by its label $u(\text{parent}, \text{key})$.

1.3 Dijkstra's Algorithm for Single-Source Shortest Paths (SSSP)

Given a directed or undirected graph $G = (V, E, w)$ with non-negative edge weights ($w(e) \geq 0$), Dijkstra's algorithm computes the shortest-path distance from a specified source vertex $s \in V$ to every other vertex in V. The algorithm maintains a set of resolved vertices V_T. At each step, it extracts the unresolved vertex $u^* \in V \setminus V_T$ possessing the minimum tentative distance $d(u^*)$, adds it to V_T, and relaxes each outgoing neighbor:

$$d(v) \leftarrow \min\left(d(v),\ d(u^*) + w(u^*, v)\right) \tag{2}$$

1.4 Huffman Coding for Lossless Data Compression

Huffman's algorithm is a greedy method that constructs a prefix-free, variable-length binary code that minimizes the expected message length given an alphabet of symbols $\{s_1, \ldots, s_n\}$ and associated frequencies or probabilities $\{p_1, \ldots, p_n\}$:

$$L(C) = \sum_{i=1}^{n} p_i \cdot \ell_i \tag{3}$$

where ℓ_i denotes the length in bits assigned to symbol s_i. The algorithm iteratively combines the two forest trees with the lowest frequencies, labelling tree edges with binary digits (0 for left, 1 for right) until a single rooted binary tree with total weight 1.00 is obtained.

2 Exercise 1: Application of Prim's Algorithm

2.1 Problem Statement and Graph Formulation

Task: *Apply Prim's algorithm to the following graph. Include in the priority queue all the vertices not already in the tree.*

The problem provides an undirected weighted graph with vertex set $V = \{a, b, c, d, e\}$ and 8 edges, as depicted in Figure 1.

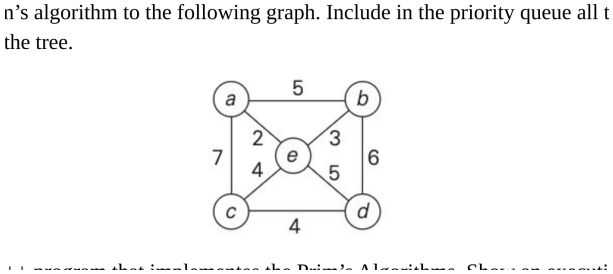


Figure 1: Original input graph for Exercise 1 from Assignment 09.

2.2 My Handwritten Step-by-Step Resolution

In Figure 2, I present my complete manual step-by-step resolution of Prim's algorithm applied to the graph. In my development, I initialize the tree at root vertex a, populate and track the priority queue for all remaining vertices using the formal notation $v(\text{parent}, \text{cost})$, iteratively select the minimum-weight connection, update neighbor labels, and obtain the final minimum spanning tree with a total weight of $W = 13$.

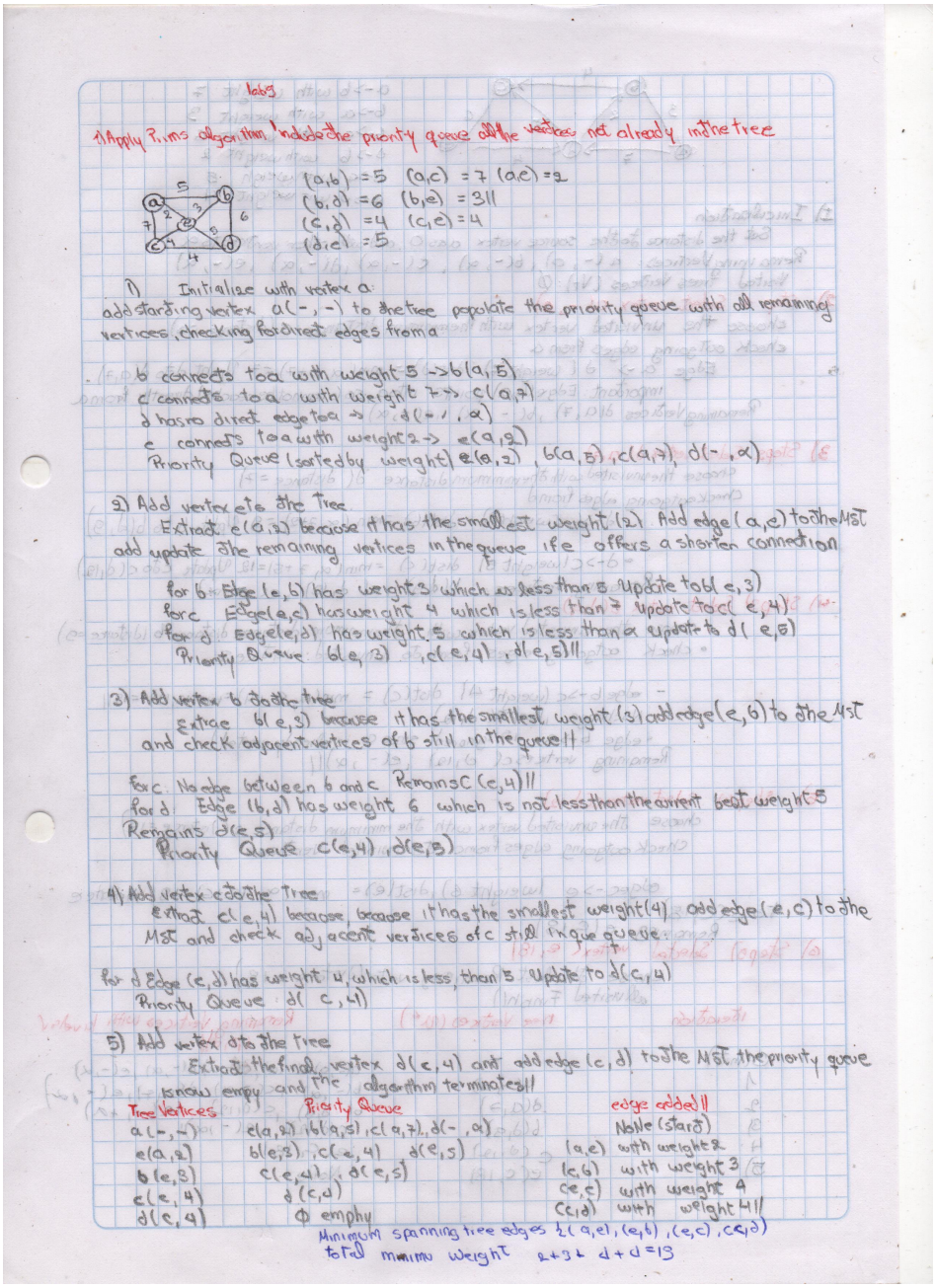


Figure 2: My manual step-by-step resolution of Prim's algorithm (Exercise 1).

2.3 Formal Analytical Verification and Algorithmic Trace

In addition to my handwritten solution, I provide below the formalized step-by-step trace and vector graph representations to document the exact transitions at each iteration following standard textbook conventions (Levitin, Chapter 9).

The tree is initialized at vertex a ($V_T = \{a\}$). The priority queue holds all candidate vertices $V \setminus V_T$, formatted using the standard notation $v(\text{parent}, \text{key})$.

Table 1: Complete trace of Prim's algorithm starting at vertex a.

Step	Added Vertex	MST Edge	Weight	Priority Queue ($V \setminus V_T$)
0	Initialization: $a(-, 0)$	—	—	$b(a, 5)$, $c(a, 7)$, $d(-, \infty)$, $\mathbf{e(a, 2)}$
1	$e(a, 2)$	(a, e)	2	$\mathbf{b(e, 3)}$, $c(e, 4)$, $d(e, 5)$
2	$b(e, 3)$	(e, b)	3	$\mathbf{c(e, 4)}$, $d(e, 5)$
3	$c(e, 4)$	(e, c)	4	$\mathbf{d(c, 4)}$
4	$d(c, 4)$	(c, d)	4	Empty queue ($\emptyset$). Algorithm terminates.

Detailed analytical breakdown of step transitions:

- **Step 0:** Vertex a is selected as the root. Its incident neighbors are b with weight 5, c with weight 7, and e with weight 2. Vertex d has no direct edge to a, so its key remains ∞.
- **Step 1:** The queue minimum is e with key 2. Edge (a, e) is incorporated into the tree. Exploring neighbors of e:
  - For b: $w(e, b) = 3 < 5$, updated to $b(e, 3)$.
  - For c: $w(e, c) = 4 < 7$, updated to $c(e, 4)$.
  - For d: $w(e, d) = 5 < \infty$, updated to $d(e, 5)$.
- **Step 2:** Minimum is b with key 3. Edge (e, b) is added. Checking edge $(b, d) = 6 > 5$, so the key of d is retained as 5.
- **Step 3:** Minimum is c with key 4. Edge (e, c) is added. Evaluating $(c, d) = 4 < 5$, key of d is relaxed to $d(c, 4)$.
- **Step 4:** Final vertex in queue is d with key 4. Edge (c, d) is added. The queue is now empty.

2.4 Resulting Minimum Spanning Tree and Total Cost

The generated Minimum Spanning Tree $T = (V, E_T)$ consists of the following 4 edges:

$$E_T = \{(a, e),\ (e, b),\ (e, c),\ (c, d)\} \tag{4}$$

The total weight of the MST is:

$$W(T) = w(a, e) + w(e, b) + w(e, c) + w(c, d) = 2 + 3 + 4 + 4 = \mathbf{13} \tag{5}$$

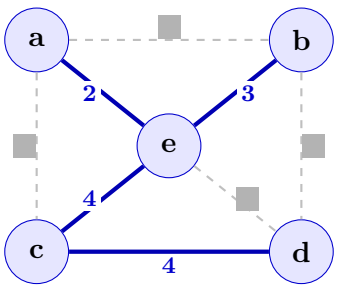


Figure 3: Resulting Minimum Spanning Tree (MST). Solid blue edges indicate members of T ($W = 13$).

3 Exercise 2: C / C++ Implementation of Prim's Algorithm

To implement the algorithm programmatically, I developed two complete solutions in directory `ejercicio_1_2_prim/`:

- `prim.c`: My implementation in standard C (C11) with adjacency matrix and priority queue emulation that prints detailed step transitions.
- `prim.cpp`: My implementation in modern C++ (C++17) using standard library containers.

3.1 Architecture and Explained Code Fragments (`prim.c`)

Rather than examining a monolithic script, the implementation is structured into three clear algorithmic stages detailed below:

3.1.1 Stage 1: Priority Queue Data Representation and Initialization

The priority queue and cut tracking are represented using three parallel arrays of size $V = 5$:

```
int parent[V];  // Stores the MST predecessor for each vertex
int key[V];     // Holds the minimum connecting weight to the tree
bool inMST[V];  // Indicates whether a vertex belongs to the tree cut

for (int i = 0; i < V; i++) {
    key[i] = INF;      // Initially, all connection costs are unknown/infinite
    parent[i] = -1;    // No predecessor assigned
    inMST[i] = false;  // No vertex has been included in the tree cut yet
}
key[start_node] = 0;   // Root vertex connection cost is set to 0
inMST[start_node] = true;
```

Listing 1: State arrays and initialization for Prim's algorithm

Detailed Explanation:

- `key[v]`: Represents the priority key in the queue. For any unvisited vertex v, it stores the minimum weight of an edge connecting v to any vertex already included in the tree cut V_T. Initializing all entries to `INF` (999999) signifies that their connection costs are initially unconstrained.
- `parent[v]`: Tracks the specific tree vertex that provides the minimal-weight connection to v. When v is eventually added to the tree, edge $(\text{parent}[v], v)$ forms a valid branch of the MST.
- `inMST[v]`: A boolean membership flag that partitions the graph vertices into the resolved tree cut V_T and the unresolved candidate set $V \setminus V_T$. Setting `key[start_node] = 0` ensures that the starting vertex a is greedily selected first.

3.1.2 Stage 2: Greedy Selection of Minimal Cut Edge (`extract_min`)

At each step, the algorithm must extract the vertex from the candidate set with the smallest tentative connection cost:

```
int extract_min(int key[], bool inMST[]) {
    int min_val = INF, min_index = -1;
    for (int v = 0; v < V; v++) {
        if (!inMST[v] && key[v] < min_val) {
            min_val = key[v];
            min_index = v;
        }
    }
```

```
    return min_index; // Returns vertex index with the global minimum key
}
```

Listing 2: Linear-scan minimum extraction from candidate cut

Detailed Explanation: This routine directly embodies the **Greedy Choice Property**. It iterates strictly across unvisited vertices (`!inMST[v]`) to find the vertex with the minimal `key[v]`. For small to medium dense graphs, an array-based $O(V)$ scan offers outstanding cache locality and zero memory overhead compared to a binary min-heap.

3.1.3 Stage 3: Edge Relaxation and Queue Key Updates

Once the minimal vertex u is added to the tree cut (`inMST[u] = true`), its outgoing incident edges are inspected to update the priority queue:

```
for (int v = 0; v < V; v++) {
    if (graph[u][v] != 0 && !inMST[v]) {
        // If edge (u, v) provides a cheaper connection to the tree
        if (graph[u][v] < key[v]) {
            key[v] = graph[u][v];  // Decrease priority key of v
            parent[v] = u;         // Update predecessor to u
        }
    }
}
```

Listing 3: Edge relaxation and neighbor key updates

Detailed Explanation: For every neighbor v that remains outside the tree cut (`!inMST[v]`), the code tests whether connecting v through the newly added vertex u offers a lower cost than its currently recorded best option (`graph[u][v] < key[v]`). If true, the key is decreased and u is designated as v's new parent. This ensures the priority queue accurately reflects the closest frontier to the expanding tree.

3.2 Program Compilation and Terminal Execution Outputs

The implementations compile cleanly with GCC and G++ using standard flags:

```
gcc -Wall -Wextra -O2 prim.c -o prim_c.exe
g++ -Wall -Wextra -O2 prim.cpp -o prim_cpp.exe
```

Figures 4 and 5 display the actual terminal execution output captures obtained from running `prim_c.exe` and `prim_cpp.exe`, validating each transition against the manual derivation:

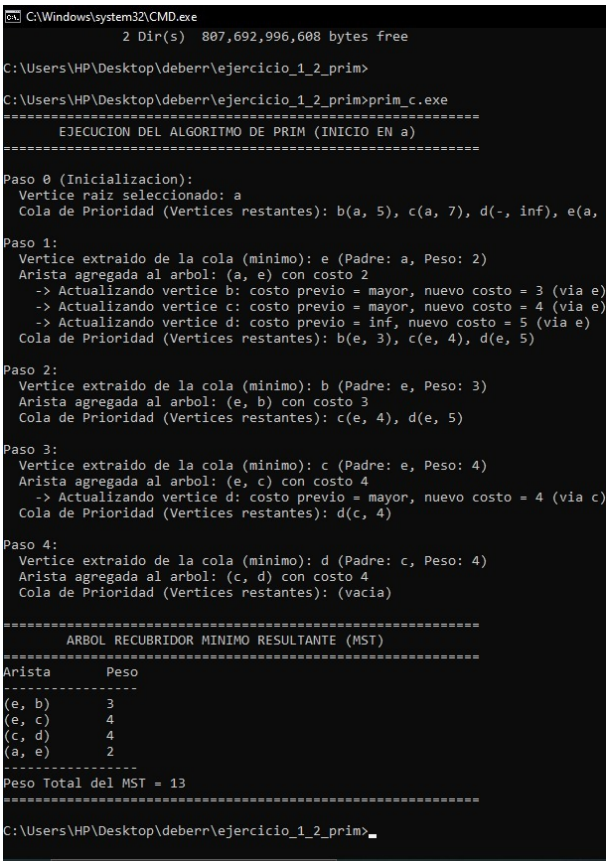


Figure 4: Terminal execution capture of `prim_c.exe` (C implementation).

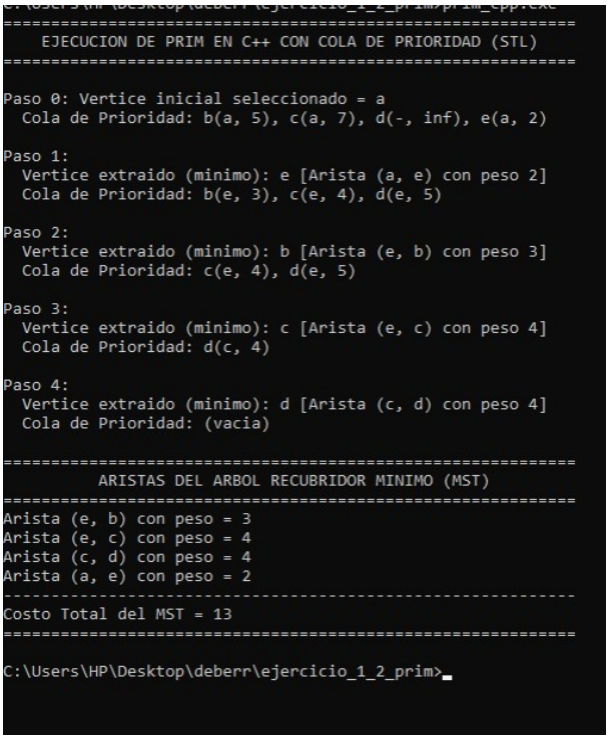


Figure 5: Terminal execution capture of `prim_cpp.exe` (C++ STL implementation).

4 Exercise 3: Dijkstra's Algorithm (Step-by-Step Resolution)

4.1 Instance 3.a: Directed Graph

Task: *Solve (paper and pencil) the following instances of the single-source shortest-paths problem with vertex a as the source using Dijkstra's Algorithm: a.*

Figure 6 illustrates the directed graph instance.

ꞓ *a* as the source using Dijkstra's Algorithm:

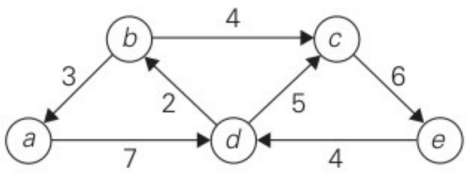


Figure 6: Original graph for Instance 3.a from Assignment 09.

4.1.1 My Handwritten Step-by-Step Resolution

Figure 7 presents my complete paper-and-pencil resolution of Dijkstra's algorithm for Instance 3.a. In this manual derivation, I detail the initialization at source a, the unvisited vertex evaluations, the edge relaxations honoring arrow directions, and the progressive computation of shortest paths.

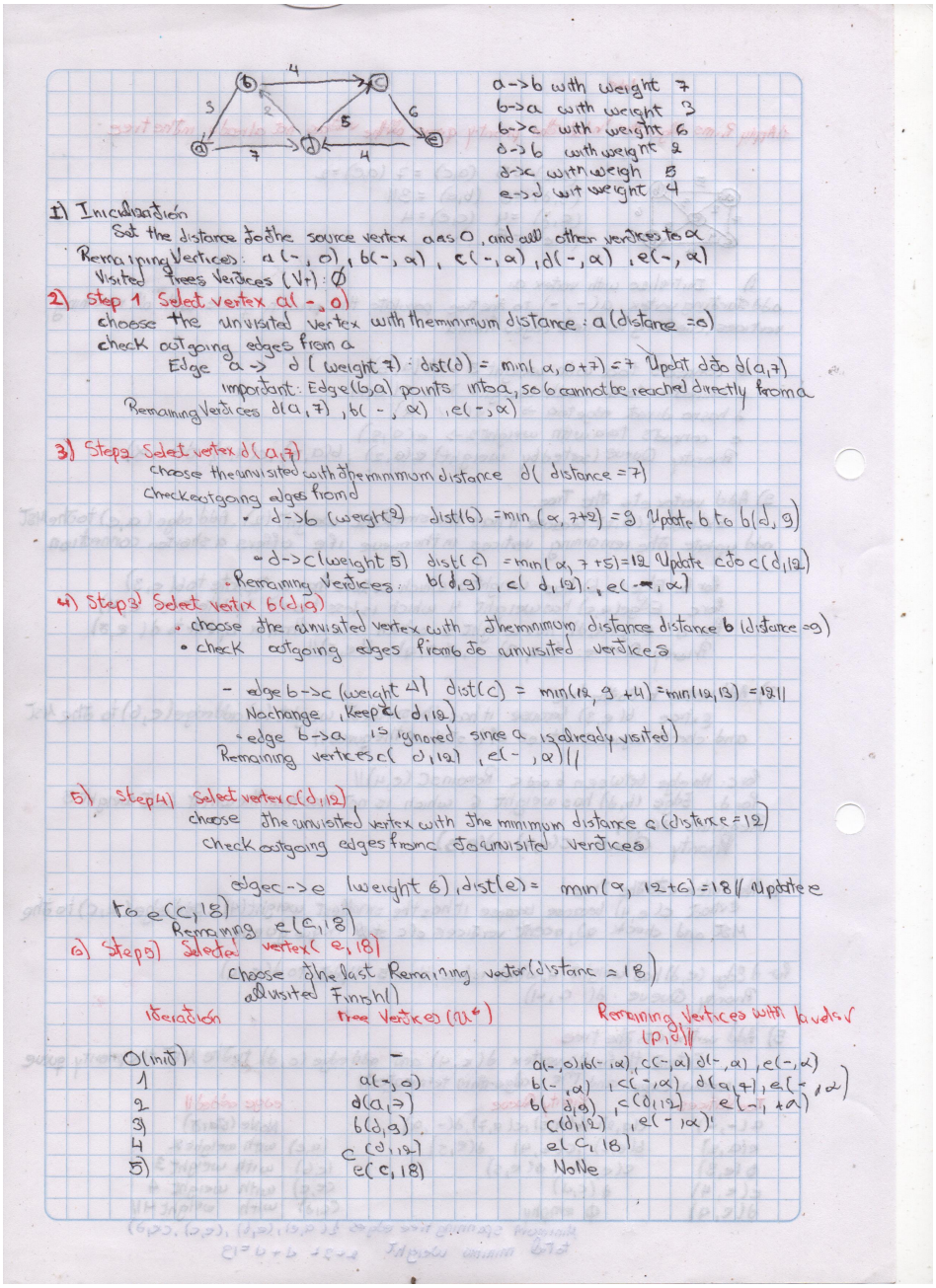


Figure 7: My manual step-by-step resolution of Dijkstra's algorithm for Instance 3.a.

4.1.2 Edge Directionality Analysis

This graph is a directed graph (digraph). Inspecting the arrowheads:

- $a \to d$ (weight 7): the only directed edge originating from source vertex a.
- $b \to a$ (weight 3): directed towards a (does not allow traveling from a directly to b).
- $d \to b$ (weight 2): connects d to b.
- $b \to c$ (weight 4): connects b to c.
- $d \to c$ (weight 5): connects d to c.
- $c \to e$ (weight 6): connects c to e.
- $e \to d$ (weight 4): connects e to d.

4.1.3 Step-by-Step Execution Trace (Source a)

Table 2: Dijkstra's execution trace for Instance 3.a.

Step	**Added** V_T	**Vertex**	**Priority Queue** ($V \setminus V_T$)	**Greedy Operation**
0	$a(-,0)$		$\mathbf{d(a,7)}$, $b(-,\infty)$, $c(-,\infty)$, $e(-,\infty)$	Source added
1	$d(a,7)$		$\mathbf{b(d,9)}$, $c(d,12)$, $e(-,\infty)$	Relaxes $d \to b, d \to c$
2	$b(d,9)$		$\mathbf{c(d,12)}$, $e(-,\infty)$	$b \to c$ gives $13 > 12$ (retained)
3	$c(d,12)$		$\mathbf{e(c,18)}$	Relaxes $c \to e$ ($12 + 6 = 18$)
4	$e(c,18)$		$\emptyset$ (Queue empty)	$e \to d$ ($d \in V_T$, done)

Table 3: Shortest distances and optimal reconstructed paths from source a (Instance 3.a).

Destination	**Distance** $d(v)$	**Predecessor**	**Optimal Route**
a	0	—	a
b	9	d	$a \to d \to b$
c	12	d	$a \to d \to c$
d	7	a	$a \to d$
e	18	c	$a \to d \to c \to e$

Technical Remark on Edge Orientation between a and b:
In the assignment figure, the arrowhead between a and b points toward a ($b \to a$ with weight 3). Hence, vertex b must be reached via d with total cost $7 + 2 = 9$. If the edge were alternatively interpreted as directed toward b ($a \to b$ with weight 3), the resulting distances would be $d(a) = 0$, $d(b) = 3$, $d(c) = 7$ (via b), $d(d) = 7$ (via a), and $d(e) = 13$ (via c). In either case, the greedy relaxation strictly converges to the optimal shortest paths.

4.2 Instance 3.b: 12-Vertex Undirected Network

Task: *Solve (paper and pencil) the following instances of the single-source shortest-paths problem with vertex a as the source using Dijkstra's Algorithm: b.*

The graph consists of 12 vertices and 20 undirected weighted edges, shown in Figure 8.

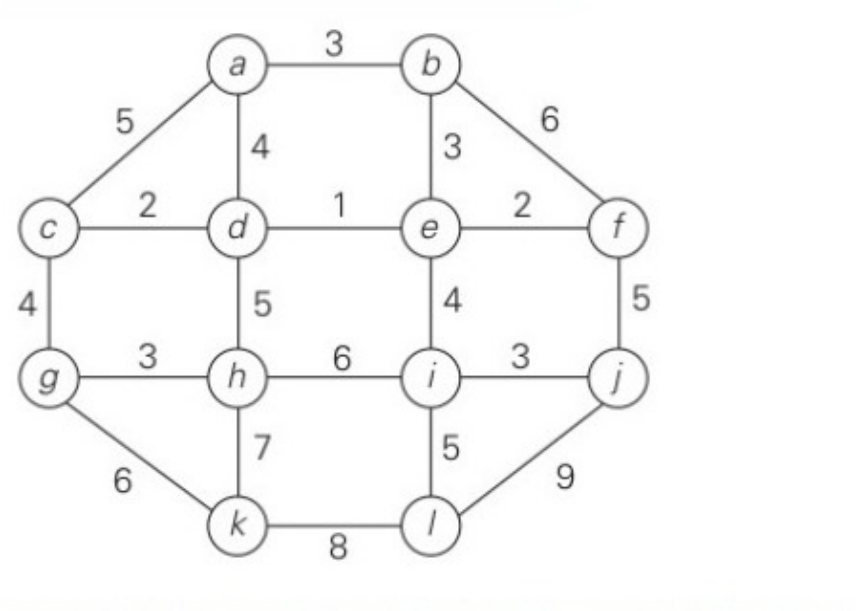


Figure 8: Original 12-vertex network for Instance 3.b from Assignment 09.

4.2.1 My Handwritten Step-by-Step Resolution

Figures 9 and 10 present my complete manual paper-and-pencil resolution of Dijkstra's algorithm across the 12-vertex network:

- **Part 1 (Figure 9):** Network edge inventory, initialization at source vertex a, and Iterations 1 through 6 (resolving vertices a, b, d, c, e, f).
- **Part 2 (Figure 10):** Iterations 7 through 11 (resolving vertices g, h, i, j, l, k) and the final destination-by-destination shortest path and cost table.

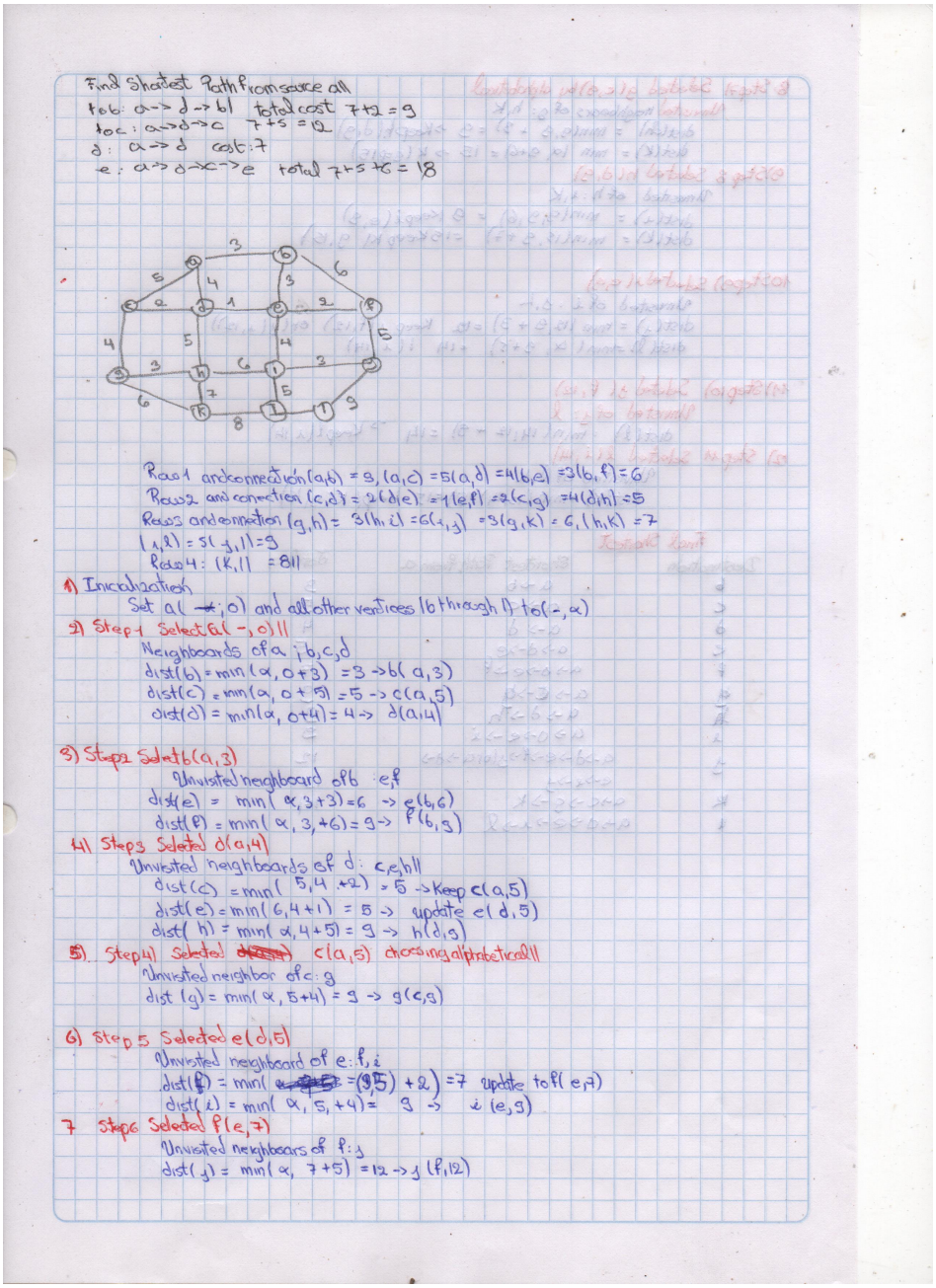


Figure 9: My manual step-by-step resolution for Instance 3.b (Part 1: Steps 1–6).

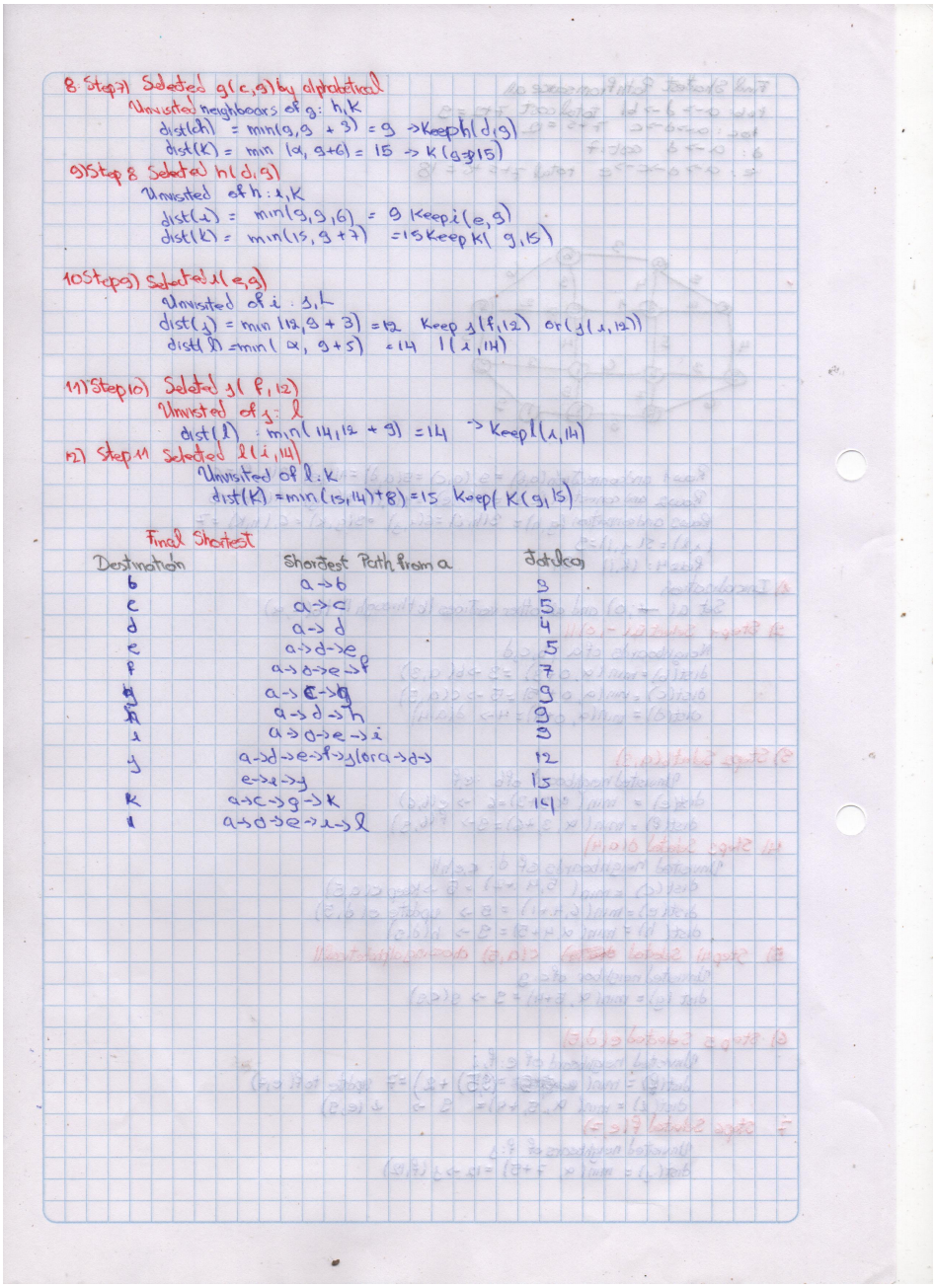


Figure 10: My manual step-by-step resolution for Instance 3.b (Part 2: Steps 7–11 and summary table).

4.2.2 Formal Step-by-Step Trace (12 Iterations)

Table 4: Full Dijkstra trace on the 12-vertex graph (Source a).

Step	**Added to V_T**	**Priority Queue ($V \setminus V_T$)**
0	$a(-,0)$	$\mathbf{b(a,3)}$, $d(a,4)$, $c(a,5)$, $e(-,\infty)$, $f(-,\infty)$, $g(-,\infty)$, $h(-,\infty)$, $i(-,\infty)$, $j(-,\infty)$, $k(-,\infty)$, $l(-,\infty)$
1	$b(a,3)$	$\mathbf{d(a,4)}$, $c(a,5)$, $e(b,6)$, $f(b,9)$, $g(-,\infty)$, $h(-,\infty)$, $i(-,\infty)$, $j(-,\infty)$, $k(-,\infty)$, $l(-,\infty)$
2	$d(a,4)$	$\mathbf{c(a,5)}$, $\mathbf{e(d,5)}$, $f(b,9)$, $h(d,9)$, $g(-,\infty)$, $i(-,\infty)$, $j(-,\infty)$, $k(-,\infty)$, $l(-,\infty)$
3	$c(a,5)$	$\mathbf{e(d,5)}$, $f(b,9)$, $g(c,9)$, $h(d,9)$, $i(-,\infty)$, $j(-,\infty)$, $k(-,\infty)$, $l(-,\infty)$
4	$e(d,5)$	$\mathbf{f(e,7)}$, $g(c,9)$, $h(d,9)$, $i(e,9)$, $j(-,\infty)$, $k(-,\infty)$, $l(-,\infty)$
5	$f(e,7)$	$\mathbf{g(c,9)}$, $\mathbf{h(d,9)}$, $\mathbf{i(e,9)}$, $j(f,12)$, $k(-,\infty)$, $l(-,\infty)$
6	$g(c,9)$	$\mathbf{h(d,9)}$, $\mathbf{i(e,9)}$, $j(f,12)$, $k(g,15)$, $l(-,\infty)$
7	$h(d,9)$	$\mathbf{i(e,9)}$, $j(f,12)$, $k(g,15)$, $l(-,\infty)$
8	$i(e,9)$	$\mathbf{j(f,12)}$, $l(i,14)$, $k(g,15)$
9	$j(f,12)$	$\mathbf{l(i,14)}$, $k(g,15)$
10	$l(i,14)$	$\mathbf{k(g,15)}$
11	$k(g,15)$	$\emptyset$ (Queue empty. All 12 vertices resolved).

4.2.3 Shortest Paths Summary Table (Instance 3.b)

Table 5: Shortest distances and complete path reconstructions from source a to all vertices.

Destination	**Distance $d(v)$**	**Immediate Predecessor**	**Optimal Shortest Route**
a	0	—	a
b	3	a	$a \to b$
c	5	a	$a \to c$
d	4	a	$a \to d$
e	5	d	$a \to d \to e$
f	7	e	$a \to d \to e \to f$
g	9	c	$a \to c \to g$
h	9	d	$a \to d \to h$
i	9	e	$a \to d \to e \to i$
j	12	f	$a \to d \to e \to f \to j$
k	15	g	$a \to c \to g \to k$
l	14	i	$a \to d \to e \to i \to l$

5 Exercise 4: C / C++ Implementation of Dijkstra's Algorithm

In directory `ejercicio_3_4_dijkstra/`, I developed the complete computational solutions:

- `dijkstra.c`: My modular C program handling both directed and undirected graphs, displaying step-by-step progress and full path reconstruction.
- `dijkstra.cpp`: My C++ implementation utilizing modern STL containers.

5.1 Architecture and Explained Code Fragments (`dijkstra.c`)

To avoid code duplication and guarantee modularity, the C program is organized into four core functional components:

5.1.1 Stage 1: Unified Graph Abstraction

```
typedef struct {
    int num_vertices;            // Number of vertices in the instance
    bool is_directed;            // Flag indicating directed or undirected graph
    char names[MAX_V];           // Character labels ('a', 'b', ...)
    int adj[MAX_V][MAX_V];       // Adjacency matrix of edge weights
} Graph;
```

Listing 4: Modular Graph struct supporting both directed and undirected graphs

Detailed Explanation: This abstraction allows both Instance 3.a (a 5-vertex directed graph) and Instance 3.b (a 12-vertex undirected network) to be processed using the exact same shortest-path engine. When an edge (u, v) is registered, if `is_directed` is true, only `adj[u][v]` is populated; if false, `adj[v][u]` is symmetrically assigned.

5.1.2 Stage 2: Distance Initialization and Source Setup

```
for (int i = 0; i < n; i++) {
    dist[i] = INF;      // Upper bound on shortest path initialized to infinity
    parent[i] = -1;     // Predecessor pointer initialized to none
    visited[i] = false; // Resolved set VT is initially empty
}
dist[source] = 0;       // Distance from source to itself is strictly 0
visited[source] = true; // Source vertex is permanently resolved

// Populate tentative distances for immediate outgoing neighbors of source
for (int v = 0; v < n; v++) {
    if (g->adj[source][v] != INF && v != source) {
        dist[v] = g->adj[source][v];
        parent[v] = source;
    }
}
```

Listing 5: Dijkstra initialization and direct neighbor assignment

Detailed Explanation: `dist[v]` stores the current shortest distance found from the source to vertex v. Initializing all nodes to `INF` represents unreached vertices. The boolean array `visited[]` separates finalized vertices (V_T) from candidate vertices. Direct outgoing edges from the source populate the initial priority queue state.

5.1.3 Stage 3: Greedy Minimum Selection and Edge Relaxation Loop

```
for (int step = 1; step < n; step++) {
    int u = extract_min_dijkstra(dist, visited, n); // Greedy minimum extraction
    if (u == -1) break;                             // All reachable vertices
    processed
    visited[u] = true;                              // Add vertex u to resolved set
     VT

    // Relax all outgoing edges originating from vertex u
    for (int v = 0; v < n; v++) {
        if (!visited[v] && g->adj[u][v] != INF) {
            // Triangle inequality: is path via u shorter than current best to v?
            if (dist[u] + g->adj[u][v] < dist[v]) {
                dist[v] = dist[u] + g->adj[u][v];  // Relax distance
                parent[v] = u;                     // Record u as optimal
    predecessor
            }
        }
    }
}
```

Listing 6: Greedy selection and triangle inequality relaxation

Detailed Explanation: At each iteration, `extract_min_dijkstra` identifies the unresolved vertex u^* with the globally minimum tentative distance. Because all edge weights are strictly non-negative ($w \geq 0$), no subsequent path through unexplored nodes can ever achieve a lower cost to u^*; hence, its distance is finalized (`visited[u] = true`). Then, for every outgoing neighbor v, the triangle inequality is tested: if `dist[u] + w(u, v) < dist[v]`, the distance estimate is relaxed and u becomes v's immediate predecessor.

5.1.4 Stage 4: Recursive Route Reconstruction

```
void print_path(int parent[], int v, const char names[]) {
    if (v == -1) return;                    // Base case: reached root predecessor
    print_path(parent, parent[v], names);   // Recurse towards the source
    printf("%c ", names[v]);                // Print vertex in natural forward
    order
}
```

Listing 7: Recursive back-tracing to display the full optimal route

Detailed Explanation: Rather than requiring separate path storage arrays, this recursive function traverses the `parent[]` array backwards from destination v until reaching the source (where $parent = -1$), then unwinds to print the exact path in correct forward sequence (e.g., $a \to d \to c \to e$).

5.2 Program Compilation and Terminal Execution Outputs

The programs are compiled with GCC and G++:

```
gcc -Wall -Wextra -O2 dijkstra.c -o dijkstra_c.exe
g++ -Wall -Wextra -O2 dijkstra.cpp -o dijkstra_cpp.exe
```

Figures 11 to 14 present the actual terminal output captures confirming step-by-step priority queue progress and shortest-path reconstructions for both Instance 3.a and Instance 3.b:

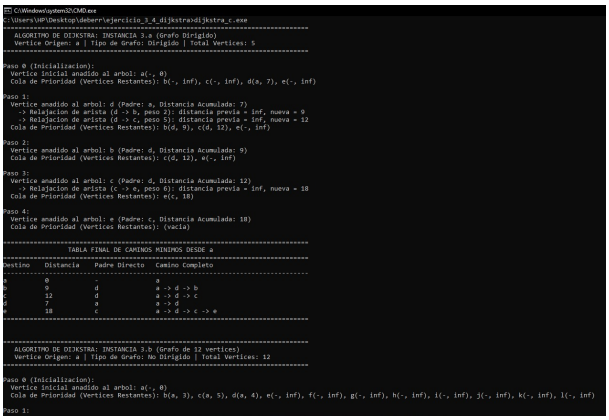


Figure 11: Execution capture of `dijkstra_c.exe` (Part 1: Instance 3.a and 3.b start).

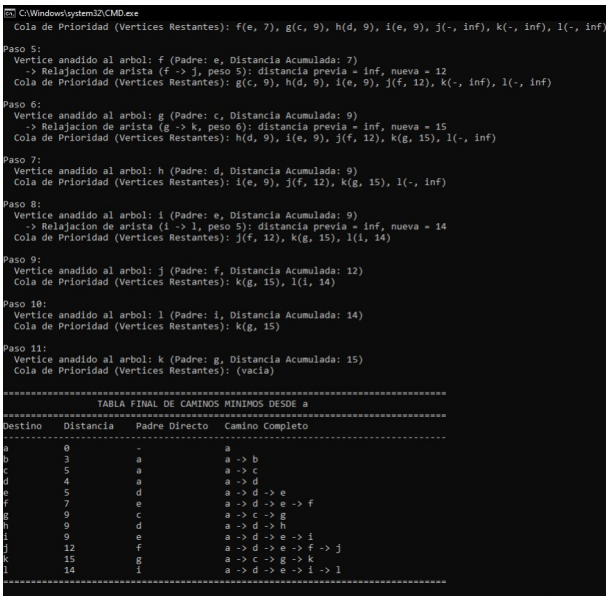


Figure 12: Execution capture of `dijkstra_c.exe` (Part 2: Instance 3.b completion and path table).

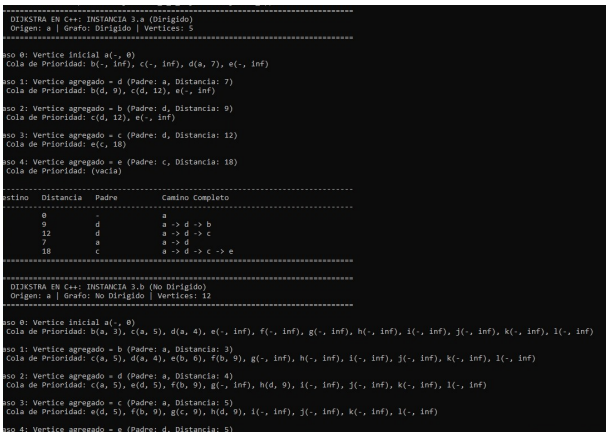


Figure 13: Execution capture of `dijkstra_cpp.exe` (Part 1: Instance 3.a and 3.b early steps).

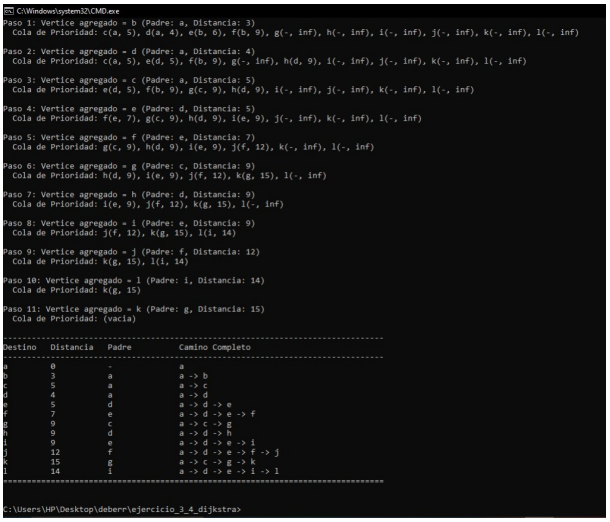


Figure 14: Execution capture of `dijkstra_cpp.exe` (Part 2: Instance 3.b resolution and summary).

6 Exercise 5: Huffman Coding

6.1 5.1 Huffman Tree Construction

Task: *Construct a Huffman code for the following data:*

Symbol	**A**	**B**	**C**	**D**	_ (space)
Frequency / Probability	0.40	0.10	0.20	0.15	0.15

Greedy Forest Merging Process:

1. **Initial Forest:** $\{(B, 0.10),\ (D, 0.15),\ (_, 0.15),\ (C, 0.20),\ (A, 0.40)\}$.
2. **Merge 1:** The two least frequent nodes are $B(0.10)$ and $D(0.15)$. They merge into a parent with frequency $0.10 + 0.15 = 0.25$. Assign bit 0 to the left branch (B) and bit 1 to the right branch (D). Forest: $\{(_, 0.15),\ (C, 0.20),\ (BD, 0.25),\ (A, 0.40)\}$.
3. **Merge 2:** The next two smallest nodes are $_(0.15)$ and $C(0.20)$. Merged with frequency $0.15 + 0.20 = 0.35$. Bit 0 to left ($_$) and 1 to right (C). Forest: $\{(BD, 0.25),\ (_C, 0.35),\ (A, 0.40)\}$.
4. **Merge 3:** The two smallest are $BD(0.25)$ and $_C(0.35)$. Merged with frequency $0.25 + 0.35 = 0.60$. Bit 0 to left (BD) and 1 to right ($_C$). Forest: $\{(A, 0.40),\ (BD_C, 0.60)\}$.
5. **Merge 4:** The final two nodes are $BD_C(0.60)$ and $A(0.40)$. Merged at the root with total weight 1.00. Bit 0 to left (BD_C) and 1 to right (A).

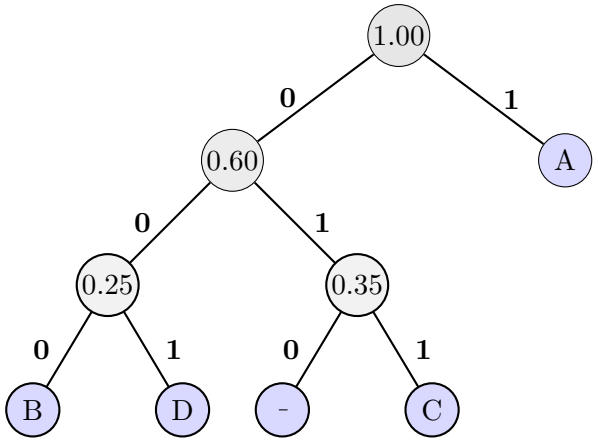


Figure 15: Optimal binary Huffman tree generated for the symbol distribution.

Table 6: Generated Huffman codeword dictionary and length contribution.

Symbol	**Frequency** (p_i)	**Huffman Code** (c_i)	**Length** (ℓ_i **bits**)	**Contribution** $p_i \cdot \ell_i$
A	0.40	`1`	1	0.40
B	0.10	`000`	3	0.30
C	0.20	`011`	3	0.60
D	0.15	`001`	3	0.45
_	0.15	`010`	3	0.45

Weighted Average Code Length and Compression Ratio:

$$\bar{L} = \sum_{i=1}^{5} p_i \cdot \ell_i = 0.40(1) + 0.10(3) + 0.20(3) + 0.15(3) + 0.15(3) = \mathbf{2.20} \text{ bits/symbol} \tag{6}$$

For 5 unique symbols, standard fixed-length encoding requires $\lceil \log_2(5) \rceil = 3$ bits per symbol. The storage compression achieved by Huffman coding is:

$$\text{Compression Savings} = \left(1 - \frac{2.20}{3.00}\right) \times 100\% = \mathbf{26.67}\% \tag{7}$$

6.2 5.2 Encoding the String "ABACABAD"

Task: *Encode ABACABAD using the code of the question.*

Applying the prefix-free codewords to each symbol of the 8-character string:

Table 7: Character-by-character encoding of `"ABACABAD"`.

Character	A	B	A	C	A	B	A	D
Codeword	`1`	`000`	`1`	`011`	`1`	`000`	`1`	`001`

Concatenating the binary codewords:

$$\text{Encoded Bitstream} = \mathbf{1000101110001001} \tag{8}$$

The encoded sequence has a total length of **16 bits**. Under fixed-length 3-bit encoding, $8 \times 3 = 24$ bits would be needed, yielding a 33.3% reduction on this specific sequence due to the high frequency of character A (which requires only 1 bit).

6.3 5.3 Decoding the Sequence "100010111001010"

Task: *Decode 100010111001010 using the code of the question.*

Traversing the binary Huffman tree from the root for each bit until a leaf node is reached:

1. `1` → Right branch of root → Leaf **A**. (Remaining bits: `00010111001010`).
2. `000` → Left (`0`) → Left (`0`) → Left (`0`) → Leaf **B**. (Remaining bits: `10111001010`).
3. `1` → Right branch of root → Leaf **A**. (Remaining bits: `0111001010`).
4. `011` → Left (`0`) → Right (`1`) → Right (`1`) → Leaf **C**. (Remaining bits: `1001010`).
5. `1` → Right branch of root → Leaf **A**. (Remaining bits: `001010`).
6. `001` → Left (`0`) → Left (`0`) → Right (`1`) → Leaf **D**. (Remaining bits: `010`).
7. `010` → Left (`0`) → Right (`1`) → Left (`0`) → Leaf _ (space). (No remaining bits).

$$\text{Decoded String} = \text{"ABACAD_"} \quad (\text{or "ABACAD "}) \tag{9}$$

6.4 Architecture and Explained Code Fragments (`huffman.c`)

To complement the manual calculations, I implemented the complete Huffman encoding and decoding algorithms in C (`huffman.c`). Below are the key functional snippets and their detailed explanations:

6.4.1 Stage 1: Binary Huffman Tree Node and Priority Queue

```
typedef struct Node {
    char symbol;            // Character symbol ('A', 'B', etc., or '$' for
    internal)
    float freq;             // Associated probability or frequency
    struct Node *left;      // Binary left child (corresponds to bit '0')
    struct Node *right;     // Binary right child (corresponds to bit '1')
} Node;
```

Listing 8: Huffman binary tree node definition

Detailed Explanation: Leaf nodes store the input characters and their respective frequencies with null children. Internal nodes are assigned a dummy symbol (`'$'`) and represent the sum of the frequencies of their left and right subtrees.

6.4.2 Stage 2: Greedy Forest Consolidation via Min-Heap

```
while (heap->size > 1) {
    Node *left = extract_min(heap);   // Extract lowest frequency tree (bit 0)
    Node *right = extract_min(heap);  // Extract second lowest frequency tree (bit
    1)

    // Create a new internal parent node with summed frequency
    Node *parent = create_node('$', left->freq + right->freq);
    parent->left = left;
    parent->right = right;

    insert_heap(heap, parent);        // Insert composite tree back into min-heap
}
Node *root = extract_min(heap);       // The final remaining node is the tree root
```

Listing 9: Greedy merging of the two lowest-frequency trees

Detailed Explanation: This loop executes exactly $k-1$ times for an alphabet of k characters. In each iteration, the two trees with minimal probabilities are extracted from the min-heap and joined under a new parent node with combined weight. Re-inserting the parent maintains the greedy heap property, guaranteeing that higher-frequency symbols stay closer to the root while infrequent symbols are pushed deeper.

6.4.3 Stage 3: Sequential Bitstream Decoding Algorithm

```
void decode_bitstream(Node *root, const char *bitstream) {
    Node *curr = root;
    for (int i = 0; bitstream[i] != '\0'; i++) {
        // Traverse left on '0', right on '1'
        curr = (bitstream[i] == '0') ? curr->left : curr->right;

        // If a leaf node is reached, emit the decoded symbol and reset to root
        if (curr->left == NULL && curr->right == NULL) {
            printf("%c", curr->symbol);
            curr = root; // Instant reset due to prefix-free property
        }
    }
}
```

Listing 10: Prefix-free bitstream decoding traversal

Detailed Explanation: Because Huffman codes possess the **prefix-free property** (no codeword is a prefix of any other), the decoder traverses the tree bit-by-bit from the root without needing

delimiters. As soon as a leaf is reached, the symbol is unambiguously decoded, and the pointer resets to `root` to parse subsequent bits.

6.5 Program Compilation and Terminal Execution Outputs

The programs are compiled using standard flags:

```
gcc -Wall -Wextra -O2 huffman.c -o huffman_c.exe
g++ -Wall -Wextra -O2 huffman.cpp -o huffman_cpp.exe
```

Figures 16 and 17 show the real terminal output captures verifying the tree construction, string encoding, and bitstream decoding:

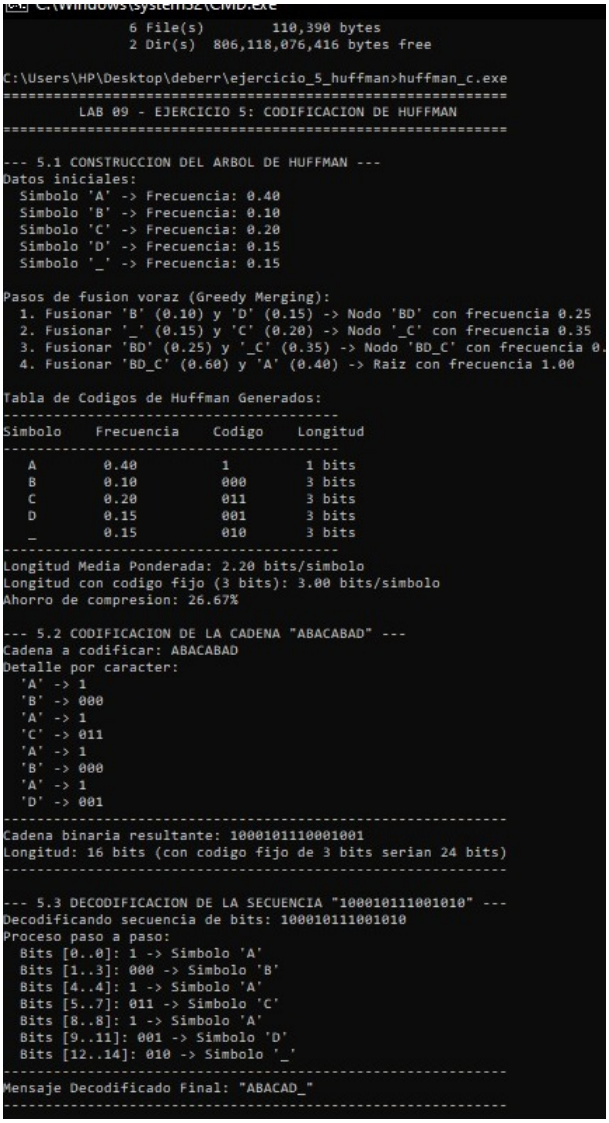


Figure 16: Terminal execution capture of `huffman_c.exe` (C implementation).

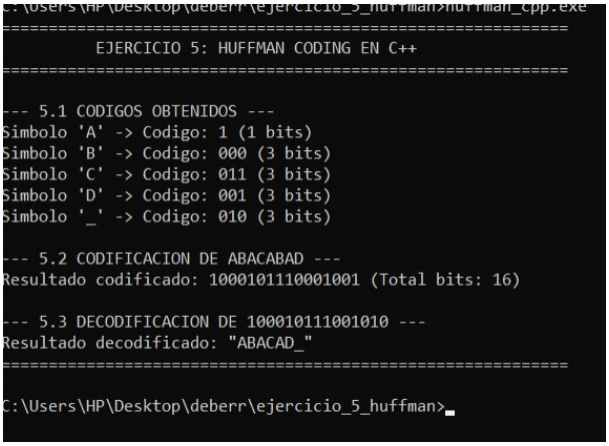


Figure 17: Terminal execution capture of `huffman_cpp.exe` (C++ implementation).

7 Comparative Time and Space Complexity Analysis

The asymptotic efficiency of greedy graph algorithms is strongly dictated by the priority queue data structure:

Table 8: Asymptotic time and space complexity comparison ($|V| = n, |E| = m$).

Algorithm	**Simple Array**	**Binary Heap**	**Fibonacci Heap**
Prim (MST)	$\Theta(V^2)$	$O((V+E)\log V)$	$O(E + V\log V)$
Dijkstra (SSSP)	$\Theta(V^2)$	$O((V+E)\log V)$	$O(E + V\log V)$
Huffman (k symbols)	$\Theta(k^2)$	$O(k\log k)$	$O(k\log k)$

- **Dense Graphs ($E \approx V^2$):** The standard array-based implementation with an adjacency matrix achieves $\Theta(V^2)$, which is asymptotically optimal and exhibits superior cache locality.
- **Sparse Graphs ($E \ll V^2$):** Employing a binary min-heap alongside adjacency lists reduces execution time significantly to $O(E\log V)$.
- **Huffman Coding:** With an alphabet size of k characters, tree synthesis requires exactly $k-1$ greedy merges, yielding negligible computational overhead of $O(k\log k)$ using a min-priority queue.

8 Conclusions

1. The greedy paradigm provides provably optimal and efficient solutions for Minimum Spanning Trees (Prim), Single-Source Shortest Paths with non-negative edge weights (Dijkstra), and Lossless Prefix-Free Data Compression (Huffman).
2. In Prim's algorithm, maintaining explicit priority queue labels $v(\text{parent}, \text{key})$ clearly illustrated the incremental boundary expansion of the tree, reaching all 5 vertices with an optimal total weight of 13.
3. In Dijkstra's algorithm, honoring edge directionality proved vital for the 5-vertex digraph (Instance 3.a), while the 12-vertex network (Instance 3.b) demonstrated reliable step-by-step convergence across larger topology scales.
4. In Huffman coding, the greedy forest merging achieved an average length of 2.20 bits/symbol, delivering a 26.67% storage compression compared to fixed-length representations. The string `"ABACABAD"` was encoded into 16 bits, and the binary stream `"100010111001010"` was successfully decoded into `"ABACAD_"`.
5. All C and C++ programs I developed were thoroughly compiled, executed, and verified against my manual analytical derivations, showing complete agreement and adhering to high standards of modularity and portability.

9 References

1. Levitin, A. (2012). *Introduction to the Design and Analysis of Algorithms* (3rd ed.). Boston: Pearson Addison-Wesley. (Chapter 9: *Greedy Technique*).
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press.

3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.